Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

Cloud Foundations & Virtualization — TP5 — Serverless Functions : développer, exécuter en local et tester une fonction Azure déclenchée par HTTP

4e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
JIDAL Ilyas
KABORE Mohammad Sharif Jonathan

Filière : Cybersécurité

Année : 2026/2027

Enseignant : M. KARRAKCHOU & M. SOUIDI

Date : 28 octobre 2026

Table des matières

Cadre du TP **2**

1 Installer l'environnement de développement **3**
1.1 Partie I.1 — Les outils . . . 3
1.2 Partie I.2 — Extension Azure Functions . . . 3

2 Créer une fonction serverless **4**
2.1 Partie II.1 — Activer le choix du modèle de programmation . . . 4
2.2 Partie II.2 — Connexion au compte Azure . . . 4
2.3 Partie II.3 — Création du projet et de la fonction . . . 5

3 Démarrer la fonction et la tester **7**
3.1 Partie III.1 — Lancement . . . 7
3.2 Partie III.2 — Sortie du terminal . . . 7
3.3 Partie III.3 — Appel depuis le navigateur . . . 8

4 Analyser la fonction par défaut **8**
4.1 Partie IV.1 — Questions sur les fichiers du projet . . . 8
4.2 Partie IV.2 — Accepter uniquement les requêtes POST . . . 9
4.3 Partie IV.3 — Relancer la fonction . . . 10
4.4 Partie IV.4 — Appel en POST avec un paramètre `name` en JSON . . . 11

5 Implémenter la fonction d'addition **12**
5.1 Partie V.1 — Code de la fonction . . . 12
5.2 Partie V.2 — Test avec le script Python . . . 13

6 Devoir : construire, exécuter et tester une fonction serverless HTTP **14**
6.1 Les étapes . . . 14
6.2 Serverless et conteneurs . . . 15

7 Bilan des éléments créés **16**

8 Difficultés rencontrées **16**

Cadre du TP

Niveau	**Cours**	**Plateformes**
4e CY	Cloud Foundations	Azure Functions Core Tools / Visual Studio Code / Python 3.14 / Microsoft Azure (abonnement *Azure for Students*)
Mode	**Approche**	**Livrable**
Groupe de 2 à 3	Développer une fonction serverless HTTP en Python, l'exécuter en local puis l'appeler depuis un script Python	Compte rendu + captures d'écran + code du script d'appel

TABLE 1 – Cadre général du TP

Objectif. Les TP précédents ont déployé des conteneurs dans Azure. Ce TP introduit un autre modèle d'exécution, le **serverless** : on n'écrit plus qu'une **fonction**, et c'est la plateforme (*Azure Functions*) qui se charge de l'exécuter quand un événement la déclenche, ici une requête HTTP. On développe une petite **API REST de calculatrice** : on crée d'abord la fonction `AddFunction` à partir du modèle *HTTP Trigger*, on l'exécute **en local**, on restreint son accès aux requêtes `POST`, puis on la transforme en véritable additionneur qui reçoit deux nombres au format JSON, avec une gestion des erreurs (code HTTP `400`). Dans chaque cas, la fonction est appelée par un script Python.

Fil conducteur du TP :

Installation des outils → Création du projet (HTTP Trigger) → Exécution locale (`func host start`) → Analyse des fichiers → Restriction à `POST` → Appel par script Python → Implémentation de l'addition avec erreurs `400`

Environnement. Poste sous Linux, éditeur Visual Studio Code avec l'extension *Azure Functions*, *Azure Functions Core Tools* version 4.16.0 (runtime 4.1054.250.26428), environnement virtuel Python 3.14.8 (`.venv`) et émulateur de stockage Azurite. Le projet est créé dans le dossier `TP5_crafting`. L'abonnement utilisé est *Azure for Students* (tenant *Université Internationale de Rabat*). Toute l'exécution de ce TP se fait en **local** : aucune ressource n'est déployée dans Azure, il n'y a donc rien à supprimer à la fin.

1 Installer l'environnement de développement

Partie I du TP : installation des outils nécessaires pour développer et exécuter une fonction Azure en Python.

1.1 Partie I.1 — Les outils

Les fonctions serverless sont développées en Python et destinées à Azure. Le sujet demande les cinq éléments suivants :

TABLE 2: Outils de l'environnement de développement

Outil	**Rôle**
Azure Functions Core Tools	Fournit la commande `func` : elle exécute le runtime Azure Functions sur la machine locale (`func host start`) et permet de tester une fonction sans la déployer
Visual Studio Code	Éditeur dans lequel la fonction est écrite, lancée et déboguée
Extension *Azure Functions* pour VS Code	Assistant de création de projet et de fonction à partir de modèles, avec connexion au compte Azure
Python 3.14	Langage de la fonction ; un environnement virtuel `.venv` lié à cette version est créé pour le projet
Politique d'exécution PowerShell	Commande `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned`, étape propre à **Windows** : elle autorise l'exécution des scripts PowerShell. Elle n'est pas nécessaire ici, le poste étant sous Linux (terminal `bash`)

1.2 Partie I.2 — Extension Azure Functions

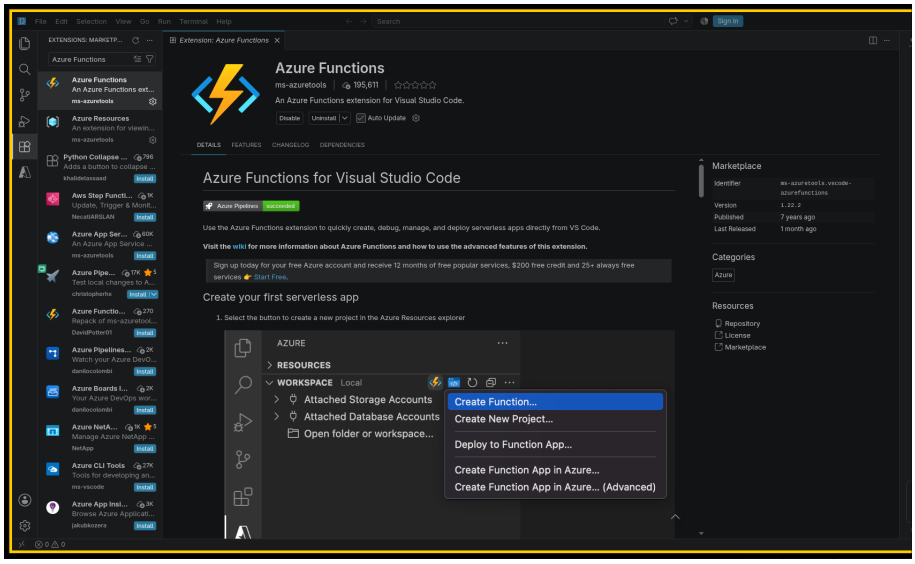


FIGURE 1 – Extension *Azure Functions* (éditeur `ms-azuretools`, identifiant `ms-azuretools.vscode-azurefunctions`, version 1.22.2) installée dans Visual Studio Code ; les boutons *Disable* et *Uninstall* confirment qu'elle est active.

2 Créer une fonction serverless

Partie II du TP : activation du choix du modèle de programmation, connexion à Azure, création du projet et de la fonction `AddFunction`.

2.1 Partie II.1 — Activer le choix du modèle de programmation

Dans *File* → *Preferences* → *Settings*, on recherche le paramètre `azureFunctions.allowProgrammingModelSelection` et on le coche. Sans lui, l'extension choisit seule le modèle de programmation Python (le plus récent) ; avec lui, elle propose de choisir, ce qui permet de prendre le **modèle V1** demandé par le sujet.

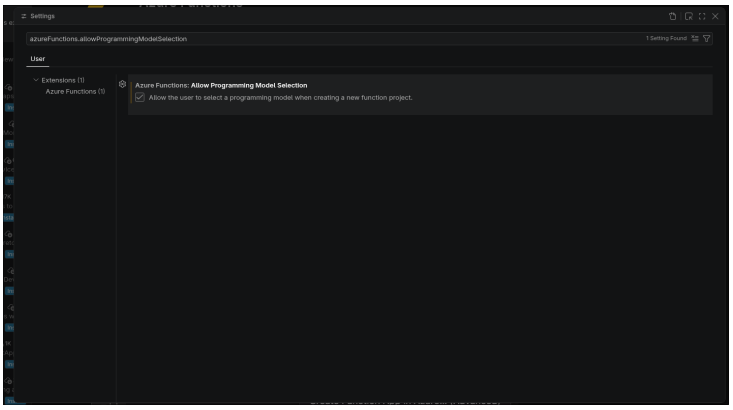


FIGURE 2 – Paramètre *Azure Functions : Allow Programming Model Selection* coché dans les réglages utilisateur.

2.2 Partie II.2 — Connexion au compte Azure

Le sujet demande de cliquer sur l'icône Azure puis sur *Sign in to Azure*, avec le compte Azure de l'UIR. Dans notre cas, les volets de l'extension (*Resources*, *Workspace*, *Accounts & Tenants*) affichaient d'abord le message *There is no data provider registered that can provide view data* (Figure 3) : l'extension n'était pas encore chargée correctement dans l'éditeur, et la fenêtre a dû être rechargée.

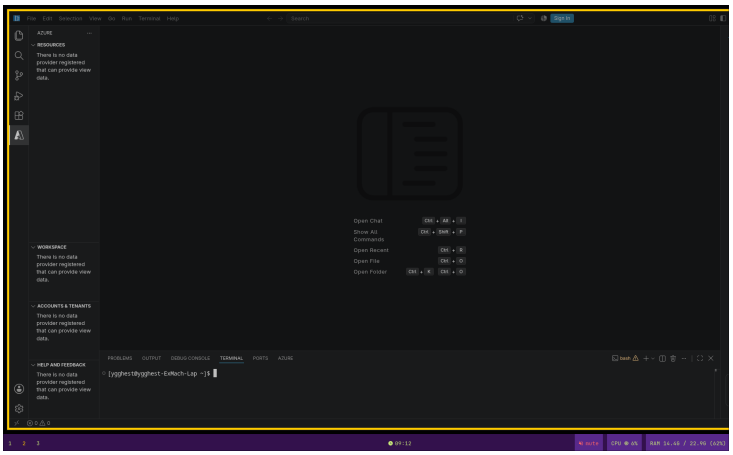


FIGURE 3 – Vue Azure de VS Code avant le rechargement : les volets *Resources*, *Workspace* et *Accounts & Tenants* n'affichent aucune donnée.

La connexion a été faite avec Azure CLI depuis le terminal intégré :

Listing 1 – Connexion à Azure

```
az login
```

La commande ouvre le navigateur ; une fois l'authentification terminée, une page locale (`localhost:41291`) confirme la connexion (Figure 4), et le terminal affiche l'abonnement *Azure for Students* du tenant *Université Internationale de Rabat*, marqué comme abonnement par défaut (Figure 5).

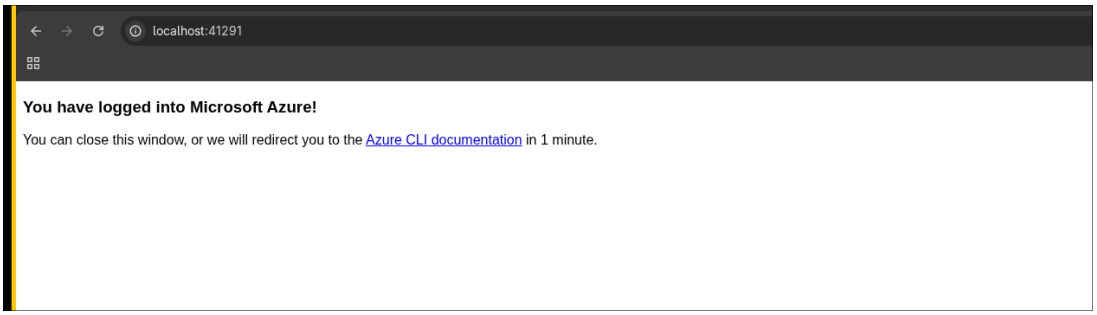


FIGURE 4 – Page de retour de l'authentification : *You have logged into Microsoft Azure !*.

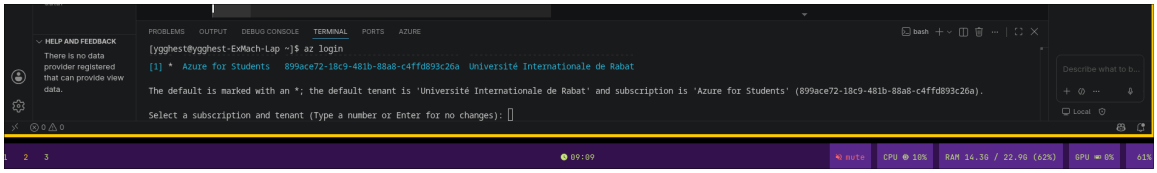


FIGURE 5 – Sortie de `az login` dans le terminal : l'abonnement *Azure for Students* est l'abonnement par défaut ; la commande propose ensuite d'en choisir un autre (Entrée pour ne rien changer).

2.3 Partie II.3 — Création du projet et de la fonction

Avec l'icône en forme d'éclair à côté de *WORKSPACE*, puis *Create New Project*, l'assistant pose une série de questions. Voici les réponses données :

TABLE 3: Paramètres de l'assistant de création

Question	**Réponse**	**Explication**
Folder	dossier du projet (`TP5_crafting`)	Dossier où est enregistré le code de la fonction (le sujet propose `calculatorFunction`)
Language	Python	Langage de la fonction
Programming model	Model V1	Modèle où chaque fonction a son dossier, avec un fichier de configuration `function.json` séparé du code

Question	Réponse	Explication
Python interpreter	Python 3.14	L'assistant crée l'environnement virtuel `.venv` avec cet interpréteur
Template	HTTP Trigger	La fonction est déclenchée par une requête HTTP
Function name	`AddFunction`	Nom de la fonction, qui devient la fin de l'URL (`/api/AddFunction`)
Authorization level	*Function*	Une clé de fonction est exigée pour l'appeler une fois la fonction déployée dans Azure
Open project	Open in current window	Le projet s'ouvre dans la fenêtre actuelle

Le code est alors généré. L'assistant installe aussi automatiquement les dépendances listées dans `requirements.txt` dans l'environnement `.venv`, comme le montre la Figure 6 de la partie suivante. Le fichier `__init__.py` généré pour le modèle V1 est le suivant :

Listing 2 – Code généré par le modèle *HTTP Trigger* (`AddFunction/__init__.py`)

```
import logging

import azure.functions as func


def main(req: func.HttpRequest) -> func.HttpResponse:
    logging.info('Python␣HTTP␣trigger␣function␣processed␣a␣request.
       ')

    name = req.params.get('name')
    if not name:
        try:
            req_body = req.get_json()
        except ValueError:
            pass
        else:
            name = req_body.get('name')

    if name:
        return func.HttpResponse(f"Hello,␣{name}.␣This␣HTTP␣
           triggered␣function␣executed␣successfully.")
    else:
        return func.HttpResponse(
             "This␣HTTP␣triggered␣function␣executed␣successfully.␣
                Pass␣a␣name␣in␣the␣query␣string␣or␣in␣the␣request␣
                body␣for␣a␣personalized␣response.",
             status_code=200
        )
```

La fonction `main` reçoit la requête (`req`) et renvoie une réponse HTTP. Elle cherche un paramètre `name` d'abord dans l'adresse (*query string*), puis dans le corps JSON ; s'il existe elle salue la personne, sinon elle renvoie un message générique.

3 Démarrer la fonction et la tester

Partie III du TP : lancement de la fonction en local et premier appel depuis le navigateur.

3.1 Partie III.1 — Lancement

Depuis l'onglet *Run and Debug*, le bouton de lecture (*Attach to Python Functions*) démarre la fonction. L'éditeur exécute deux tâches l'une après l'autre : l'installation des dépendances dans `.venv`, puis le démarrage de l'hôte Azure Functions. Cela équivaut, dans un terminal, à :

Listing 3 – Démarrage de l'hôte Azure Functions en local

```
source .venv/bin/activate
func start
```

L'émulateur de stockage **Azurite** est lancé aussi (la barre d'état affiche ses services *Queue* sur le port 10001 et *Blob* sur le port 10000). Azure Functions a en effet besoin d'un compte de stockage pour son fonctionnement interne ; en local, Azurite joue ce rôle sans avoir à créer de compte de stockage dans Azure.

3.2 Partie III.2 — Sortie du terminal

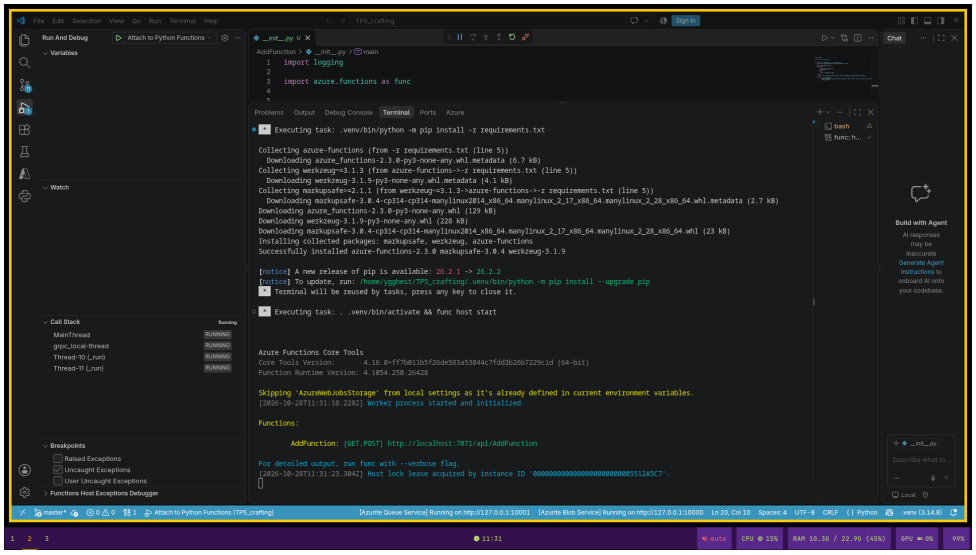


FIGURE 6 – Démarrage de la fonction dans VS Code : installation des paquets (`azure-functions`, `werkzeug`, `markupsafe`) dans `.venv`, puis `func host start`. La sortie indique *Worker process started and initialized* et la liste des fonctions : `AddFunction: [GET,POST] http://localhost:7071/api/AddFunction`.

La sortie du terminal donne les informations utiles : la version des outils (*Core Tools* 4.16.0, runtime 4.1054.250.26428), le démarrage du processus *worker* Python, et surtout la **liste des fonctions avec leur adresse** : `AddFunction` accepte à ce stade les méthodes `GET` et `POST` à l'adresse `http://localhost:7071/api/AddFunction`. L'URL se compose du port `7071` (port par défaut de l'hôte local), du préfixe `/api` et du nom de la fonction. Le message jaune sur `AzureWebJobsStorage` signale seulement que cette variable est déjà définie dans l'environnement et que la valeur de `local.settings.json` est donc ignorée ; il est sans conséquence.

3.3 Partie III.3 — Appel depuis le navigateur

Comme la fonction accepte `GET`, on peut l'appeler en ouvrant simplement l'adresse dans le navigateur (une requête `GET`, sans paramètre).

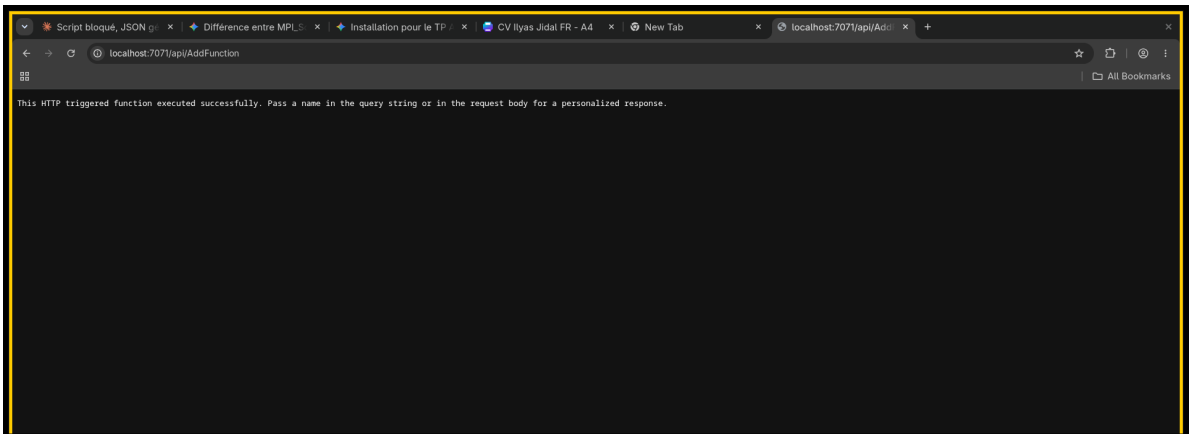


FIGURE 7 – Appel de `localhost:7071/api/AddFunction` dans le navigateur : la fonction répond avec le message générique, car aucun paramètre `name` n'a été fourni.

La fonction a bien été exécutée (statut `200`). Le message demande de passer un `name` dans l'adresse ou dans le corps de la requête pour obtenir une réponse personnalisée, ce qui correspond au code étudié plus haut. En local, la clé de fonction demandée par le niveau d'autorisation *Function* n'est pas exigée ; elle le serait une fois la fonction déployée dans Azure.

4 Analyser la fonction par défaut

Partie IV du TP : rôle des fichiers du projet, restriction aux requêtes `POST` et appel de la fonction avec un corps JSON.

4.1 Partie IV.1 — Questions sur les fichiers du projet

Le projet contient plusieurs fichiers. Les réponses aux questions du sujet sont les suivantes :

TABLE 4: Réponses aux questions de la section IV

Question	**Fichier**	**Explication**
Quel fichier contient le **code** de la fonction ?	`AddFunction/__init__.py`	Contient la fonction `main`, point d'entrée appelé à chaque requête (le nom du fichier est donné par `scriptFile` dans `function.json`)
Quel fichier contient la **configuration** de la fonction (méthodes HTTP acceptées, par exemple) ?	`AddFunction/function.json`	Déclare le déclencheur (*binding*) : type `httpTrigger`, niveau d'autorisation (`authLevel`), liste des méthodes (`methods`) et sortie HTTP
Quel fichier contient le **nom des bibliothèques Python** utilisées ?	`requirements.txt` (à la racine du projet)	Liste les paquets installés par `pip` dans l'environnement : ici `azure-functions`, qui apporte le module `azure.functions`

Deux autres fichiers de la racine complètent le projet : `host.json`, qui contient la configuration globale de l'hôte Azure Functions, et `local.settings.json`, qui contient les paramètres propres au poste local (par exemple l'emplacement du stockage), uniquement utilisés en local.

4.2 Partie IV.2 — Accepter uniquement les requêtes POST

La liste des méthodes se trouve dans `function.json`. Le fichier d'origine déclare `"methods": ["get", "post"]` ; on ne garde que `"post"` :

Listing 4 – `AddFunction/function.json` après modification

```
{
  "scriptFile": "__init__.py",
  "bindings": [
    {
      "authLevel": "function",
      "type": "httpTrigger",
      "direction": "in",
      "name": "req",
      "methods": [
        "post"
      ]
    },
    {
      "type": "http",
      "direction": "out",
      "name": "$return"
    }
  ]
```

```
}
```

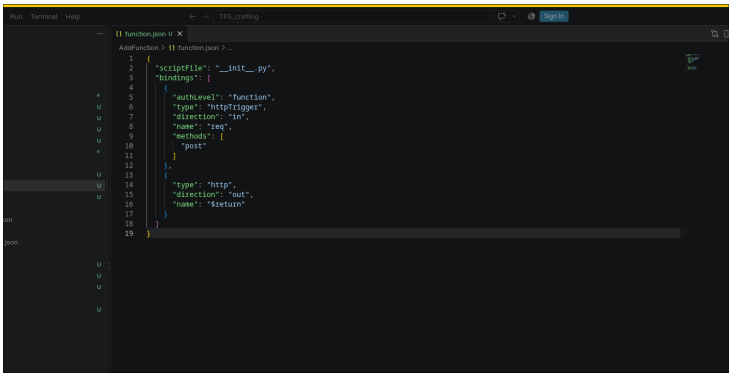


FIGURE 8 – `function.json` modifié dans l'éditeur : la liste `methods` ne contient plus que `"post"`.

Les deux *bindings* se lisent ainsi : le premier (`direction: in`) est le **déclencheur HTTP**, dont l'objet est passé à `main` sous le nom `req` ; le second (`direction: out`, `$return`) indique que la valeur retournée par `main` est la **réponse HTTP**.

4.3 Partie IV.3 — Relancer la fonction

Après avoir arrêté puis relancé l'hôte, la liste des fonctions n'indique plus que `POST` (Figure 10, partie haute) :

Listing 5 – Relance de la fonction

```
cd ~/TP5_crafting
source .venv/bin/activate
func start
```

Ouvrir de nouveau l'adresse dans le navigateur ne fonctionne alors plus : le navigateur envoie une requête `GET`, que la fonction n'accepte plus.

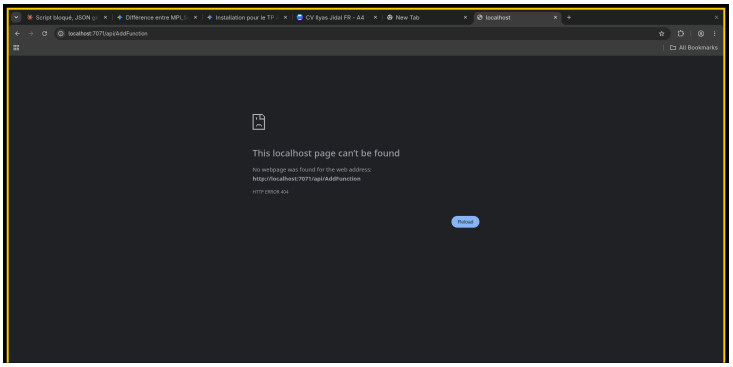


FIGURE 9 – Même adresse après la modification : *HTTP ERROR 404*.

La réponse est un code `404` (et non `405 Method Not Allowed`) : la route `/api/AddFunction` n'est plus déclarée que pour la méthode `POST`, donc pour l'hôte aucune route ne correspond à une requête `GET`. Un navigateur ne peut pas envoyer de `POST` avec un corps JSON en saisissant simplement une adresse : il faut un autre client, ici un script Python.

4.4 Partie IV.4 — Appel en POST avec un paramètre `name` en JSON

Le script reprend le principe de celui du classifieur de bananes (module `requests`, requête `POST`) en envoyant un corps JSON au lieu des octets d'une image.

Listing 6 – Script d'appel `call_function.py` (envoi de `name`)

```python
import requests

api_url = "http://localhost:7071/api/AddFunction"

# Corps de la requete : un objet JSON avec le parametre "name"
payload = {"name": "Ilyas"}

# json=payload convertit le dictionnaire en JSON
# et ajoute l'en-tete Content-Type: application/json
response = requests.post(api_url, json=payload)

print("Status code:", response.status_code)
print("Response:", response.text)
```

L'option `json=` de `requests` remplace l'écriture manuelle de `headers={"Content-Type": "application/json"}` et de `data=json.dumps(...)` : la bibliothèque convertit le dictionnaire en JSON et ajoute l'en-tête elle-même.

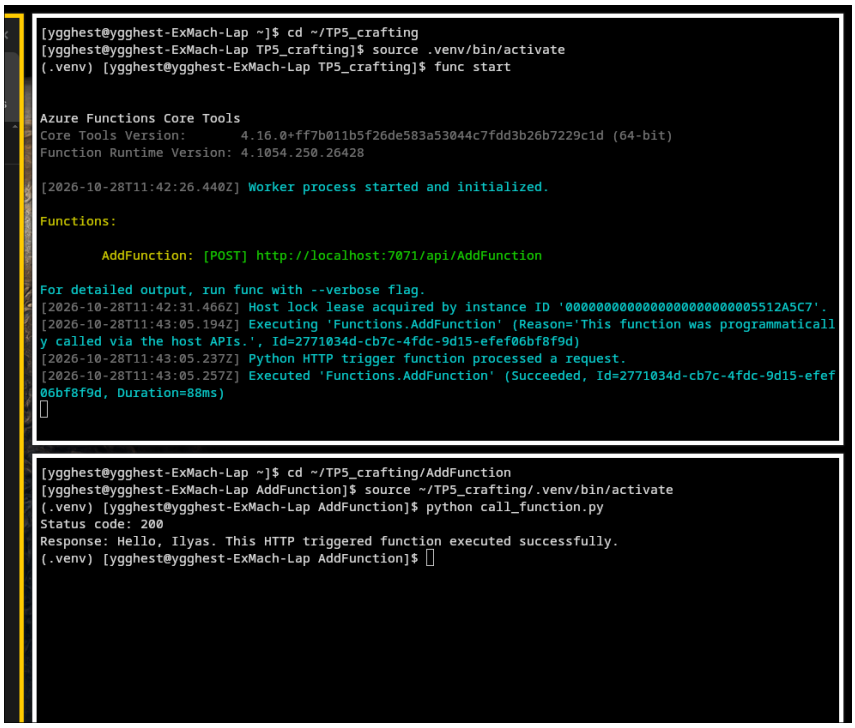


FIGURE 10 – En haut, le terminal de l'hôte : `AddFunction: [POST] http://localhost:7071/api/AddFunction`, puis les journaux de la requête (*Python HTTP trigger function processed a request*, *Executed 'Functions.AddFunction' (Succeeded, Duration=88ms)*). En bas, le script : `Status code: 200` et `Response: Hello, Ilyas. This HTTP triggered function executed successfully.`

Le code 200 et la réponse personnalisée montrent que la fonction a bien lu le paramètre name dans le corps JSON, via req.get_json(). Les journaux de l'hôte confirment l'exécution, avec une durée de 88 ms.

5 Implémenter la fonction d'addition

Partie V du TP : la fonction reçoit deux nombres en JSON et renvoie leur somme ; en cas de paramètres absents ou incorrects, elle renvoie une erreur 400.

5.1 Partie V.1 — Code de la fonction

Listing 7 – AddFunction/__init__.py : addition de deux nombres

```
import logging
import math

import azure.functions as func


def erreur(message: str) -> func.HttpResponse:
    """Renvoie une erreur HTTP 400 avec un message."""
    return func.HttpResponse(message, status_code=400)


def est_nombre(valeur) -> bool:
    """Vrai pour un entier ou un flottant fini (les booleens sont
       refuses)."""
    return (isinstance(valeur, (int, float))
            and not isinstance(valeur, bool)
            and math.isfinite(valeur))


def main(req: func.HttpRequest) -> func.HttpResponse:
    logging.info('AddFunction: requete recue.')

    # 1. Le corps doit etre un JSON valide
    try:
        body = req.get_json()
    except ValueError:
        return erreur("Erreur : le corps de la requete doit etre un
           JSON valide.")

    # 2. Le JSON doit etre un objet contenant les parametres "a" et
        "b"
    if not isinstance(body, dict) or "a" not in body or "b" not in
       body:
        return erreur("Erreur : les parametres 'a' et 'b' sont
           obligatoires.")

    a = body["a"]
```

```
    b = body["b"]

    # 3. Les deux parametres doivent etre des nombres
    if not est_nombre(a) or not est_nombre(b):
        return erreur("Erreur : 'a' et 'b' doivent etre des nombres.")

    # 4. Calcul et reponse
    a, b = float(a), float(b)
    return func.HttpResponse(f"The sum of {a} and {b} is {a + b}.", status_code=200)
```

TABLE 5: Cas gérés par la fonction

Corps de la requête	**Code**	**Résultat**
`{"a": 5, "b": 3}`	200	`The sum of 5.0 and 3.0 is 8.0.`
Corps vide ou texte qui n'est pas du JSON	400	Erreur : le corps doit être un JSON valide
`{"a": 5}` (paramètre manquant)	400	Erreur : `a` et `b` sont obligatoires
`{"a": "cinq", "b": 3}` (valeur qui n'est pas un nombre)	400	Erreur : `a` et `b` doivent être des nombres

La fonction vérifie dans l'ordre la validité du JSON, la présence des deux paramètres, puis leur type. Les booléens sont refusés explicitement, car en Python `True` est un entier (`isinstance(True, int)` vaut `True`) et serait sinon accepté comme le nombre 1. Les nombres sont convertis en flottants avant l'addition, ce qui explique l'affichage `5.0` et `3.0` dans la réponse.

5.2 Partie V.2 — Test avec le script Python

Le script d'appel est adapté pour envoyer deux nombres :

Listing 8 – Script d'appel `call_function.py` (envoi de deux nombres)

```
import requests

api_url = "http://localhost:7071/api/AddFunction"

# Deux nombres envoyes dans un corps JSON
payload = {"a": 5, "b": 3}

response = requests.post(api_url, json=payload)

print("Status code:", response.status_code)
print("Response:", response.text)
```

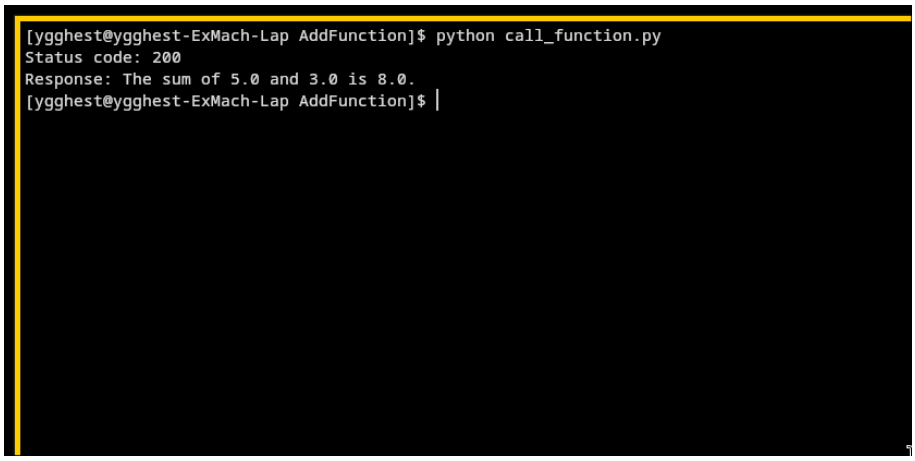


FIGURE 11 – Exécution de `python call_function.py` : `Status code: 200` et `Response: The sum of 5.0 and 3.0 is 8.0.`

La fonction renvoie bien la somme des deux nombres reçus en JSON. Pour tester les erreurs, il suffit de modifier `payload` dans le script (paramètre retiré, ou valeur de type texte) : la réponse porte alors le statut `400` et le message d'erreur correspondant, en suivant le tableau ci-dessus.

6 Devoir : construire, exécuter et tester une fonction serverless HTTP

6.1 Les étapes

1. **Installer les outils** : Azure Functions Core Tools, Visual Studio Code, l'extension *Azure Functions* et Python 3.14 (Figure 1) ; l'étape PowerShell ne concerne que Windows.
2. **Activer le choix du modèle de programmation** (`azureFunctions.allowProgrammingModelSelection`, Figure 2), puis **se connecter à Azure** (Figures 4 et 5).
3. **Créer le projet** avec l'assistant : langage Python, modèle V1, interpréteur Python 3.14, modèle *HTTP Trigger*, fonction `AddFunction`, niveau d'autorisation *Function*.
4. **Démarrer la fonction** (*Run and Debug* ou `func start`, avec l'émulateur Azurite) et relever l'adresse affichée dans le terminal (Figure 6).
5. **La tester** : appel depuis le navigateur en `GET` (Figure 7).
6. **Analyser les fichiers** `__init__.py`, `function.json` et `requirements.txt` (section IV.1).
7. **Restreindre la fonction à** `POST` dans `function.json`, puis la relancer (Figures 8 et 9).
8. **L'appeler avec un script Python** envoyant un corps JSON (Figure 10).
9. **Implémenter l'addition** avec validation des paramètres et réponse `400` en cas d'erreur, puis la tester (Figure 11).

TABLE 6: Commandes, fichiers et options utilisés dans le TP

Commande, fichier ou option	**Rôle**
`az login`	Connecte Azure CLI au compte Azure par le navigateur
`source .venv/bin/activate`	Active l'environnement virtuel Python du projet
`func start` (ou `func host start`)	Démarre l'hôte Azure Functions en local et affiche l'adresse de chaque fonction
`__init__.py`	Code de la fonction (point d'entrée `main`)
`function.json`	Configuration : déclencheur HTTP, niveau d'autorisation, méthodes acceptées, sortie
`requirements.txt`	Bibliothèques Python à installer (`azure-functions`)
`host.json` / `local.settings.json`	Configuration globale de l'hôte / paramètres propres au poste local
`authLevel: function`	Niveau d'autorisation : clé de fonction exigée dans Azure (non exigée en local)
`requests.post(url, json=...)`	Envoie une requête HTTP `POST` avec un corps JSON
`func.HttpResponse(..., status_code=400)`	Renvoie une réponse HTTP avec le code de statut choisi

6.2 Serverless et conteneurs

TABLE 7: Fonction serverless (Azure Functions) et conteneur (TP4)

Critère	**Azure Functions (ce TP)**	**Conteneur (Container Instance / App, TP4)**
Ce qu'on fournit	Uniquement le code d'une fonction et sa configuration	Une image complète (système, bibliothèques, serveur web, code)
Déclenchement	Par un événement (ici une requête HTTP) décrit dans `function.json`	Le conteneur tourne en continu et écoute sur un port
Serveur web	Fourni par la plateforme : pas de Flask à écrire	À inclure dans l'image (Flask dans le TP4)
Test en local	`func start` avec Azurite	`docker run`
Usage typique	Petites tâches courtes déclenchées par des événements, API légères	Applications complètes, services avec dépendances particulières

En résumé, avec une fonction serverless, le développeur ne gère ni image, ni serveur web, ni port : il écrit la logique métier, déclare dans `function.json` ce qui la déclenche, et la plateforme s'occupe de l'exécuter.

7 Bilan des éléments créés

Table 8: Éléments créés pendant le TP5

Élément	**Rôle**
Projet Azure Functions `TP5_crafting`	Projet Python (modèle V1) contenant la fonction, avec son environnement virtuel `.venv` (Python 3.14.8)
Fonction `AddFunction`	Fonction HTTP Trigger : d'abord la fonction de salutation par défaut, puis l'addition de deux nombres
`AddFunction/__init__.py`	Code de la fonction
`AddFunction/function.json`	Configuration : déclencheur HTTP limité à `POST`, niveau d'autorisation *Function*
`requirements.txt`	Dépendance `azure-functions`
Adresse locale `http://localhost:7071/api/AddFunction`	Point d'entrée de la fonction pendant l'exécution locale
Émulateur Azurite	Stockage local nécessaire au fonctionnement de l'hôte
Script `call_function.py`	Appelle la fonction en `POST` avec un corps JSON

8 Difficultés rencontrées

Table 9: Problèmes rencontrés et solutions

Problème	**Cause**	**Solution**
Volets Azure vides : *There is no data provider registered that can provide view data*	L'extension *Azure Functions* n'était pas encore chargée correctement dans l'éditeur	Recharger la fenêtre de l'éditeur, puis se reconnecter à Azure
Choix du modèle de programmation absent dans l'assistant	Le paramètre `azureFunctions.allowProgrammingModelSelection` était désactivé	Le cocher dans les réglages, puis relancer la création du projet
Après la restriction à `POST`, l'adresse ouverte dans le navigateur renvoie *HTTP ERROR 404*	Le navigateur envoie une requête `GET`, et la route n'est plus déclarée que pour `POST`	Appeler la fonction avec un script Python (ou `curl -X POST`)
Message *Skipping 'AzureWebJobsStorage' from local settings* au démarrage	La variable est déjà définie dans l'environnement du poste, ce qui rend la valeur de `local.settings.json` inutile	Aucune action : simple information, la fonction démarre normalement
Étape `Set-ExecutionPolicy` du sujet	Cette commande concerne PowerShell sous Windows ; le poste est sous Linux	Étape ignorée